Quiz

COMP102: Data Structures

University Mohammed VI Polytechnic — SASE

Academic Year 2025–2026

Instructions. Answer all questions on a separate sheet. For classification questions, choose all answers that apply. For tracing questions, show the state after each operation. For coding questions, write **valid** Python code. Total: **20 points**.

Question 1 — 1.5 points

For each statement, decide whether it is **True** or **False** and justify your answer in one sentence.

1. Two programs that both use the Stack ADT must store their elements the same way in memory.
2. Replacing a hash-table implementation of a Dict with a balanced BST changes the operations available in the ADT's interface.
3. A Python `list` is an abstract data type.
4. The same ADT operation can have different time complexities under two different implementations.

Question 2 — 1.5 points

(a) Which of the following values can safely be inserted into a `set` or used as keys in a `dict`? For each, briefly explain your decision.

```
()
(1,)
"hello"
tuple([5, 6])
(1, [2])
((1, 2), (3, 4))
```

(b) What is the value of the following expression, and why?

```
len({1, 1.0, True, 2, 2.0})
```

Question 3 — 1.5 points

What is printed by the following code?

```
a = [[1, 2], [3, 4]]
b = a[:]

b[0].append(99)
b.append([5, 6])

print(a)
print(b)
```

Question 4 — 2 points

For each situation, choose the best data structure or ADT and justify your choice in one sentence.

1. You must detect whether a value has already appeared while scanning a stream of items.
2. You must repeatedly remove and process the item with the highest priority.
3. You must implement the "back" button of a web browser.
4. Requests to a server must be handled strictly first-come, first-served.
5. You must map each product code to its price.
6. You must insert and delete elements frequently at the **beginning** of a sequence.
7. You must directly access the element at position `k` in a fixed list of records.
8. You must reverse the order of the elements read from an input stream.
9. You must maintain values so that search, insertion, and sorted traversal are all efficient.

Question 5 — 2 points

For each binary tree below, choose **all** classifications that apply:

BST AVL tree Min-heap Max-heap None of these

Use the following conventions:

- A BST uses the strict rule: left subtree values $<$ root $<$ right subtree values.
- An AVL tree is a BST where every node has balance factor -1, 0, or 1.

- A binary heap must satisfy the complete-tree shape property and the appropriate heap-order rule (a heap may contain duplicate values).

Tree A

```
      12
     /
    8
   /
  5
```

Tree B

```
      20
     /  \
    10   30
   /  \
  5    25
```

Tree C

```
      4
     / \
    4    7
   / \   /
  9   5 8
```

Tree D

```
      60
     /  \
    55   40
   /  \   /
  50  20 30
```

Tree E

```
      50
     /  \
    30   70
   /  \    \
  20  40    80
```

For each tree, briefly justify your classification. (Reminder: a valid BST is not automatically an AVL tree, and a complete-shaped tree is not automatically a heap.)

Question 6 **1.5 points**

A stack starts empty. Represent the stack **vertically**. Trace the stack after each operation.

```
push(7)
push(3)
pop()
push(5)
peek()
push(8)
pop()
pop()
push(2)
push(6)
pop()
peek()
```

For each step, write the operation, the returned value if any, and the full stack after the operation.

Question 7 **1.5 points**

A queue starts empty. The front of the queue is shown on the **left**. Trace the queue after each operation.

```
enqueue("x")
enqueue("y")
front()
enqueue("z")
dequeue()
enqueue("w")
dequeue()
front()
enqueue("v")
```

```
dequeue()
dequeue()
front()
```

For each step, write the operation, the returned value if any, and the full queue after the operation.

Question 8 **2.5 points**

Insert the following values into an empty AVL tree:

`50, 30, 70, 20, 40, 35, 60`

After each insertion:

1. Draw the tree.
2. Write the height of each node.
3. Write the balance factor of the first unbalanced node, if any.
4. Identify the AVL case: LL, RR, LR, or RL.
5. State which rotation or rotations are applied.

Draw the final AVL tree after all insertions.

Question 9 **2 points**

Start with an empty **min-heap**. After each operation, **draw the binary heap as a tree**.

```
push(8)
push(8)
push(3)
push(10)
push(3)
pop()
pop()
push(1)
push(6)
pop()
```

For each step, write the operation, the returned value if any, and the resulting heap drawn as a tree. Heaps may contain duplicate values; when two children are equal during a sift-down, swap with the left child.

Question 10 **2 points**

Implement a class `MyQueue`.
Required attribute:

`items`

Required methods:

`__init__`	`enqueue`	`dequeue`
`front`	`is_empty`	`size`

Rules:

- Use a Python list internally. Treat index `0` as the front of the queue.
- `enqueue` adds `value` to the back of the queue.
- `dequeue` removes and returns the front value.
- `front` returns the front value without removing it.
- `is_empty` returns `True` if the queue has no elements and `False` otherwise.
- `size` returns the number of elements in the queue.
- Calling `dequeue` or `front` on an empty queue must raise `IndexError("empty queue")`.

Question 11 **2 points**

Implement the function `power(base, exp)`.
The function returns `base` raised to the power `exp`.
Rules:

- `base` is a number and `exp` is a nonnegative integer.
- Solve the problem recursively.
- Do not use the `**` operator or the built-in `pow`.
- Do not use loops.

Examples:

```
power(2, 5)       # 32
power(5, 0)       # 1
power(3, 3)       # 27
power(7, 1)       # 7
power(10, 4)      # 10000
```

After the code, explain the base case, the smaller recursive subproblem, how the result is combined at each step, why the recursion eventually stops, the time complexity, and the recursion-stack space complexity.